Πετράκης Ραφαήλ Ιωάννης ⋆ csd5748

ΗΥ-380: 2^{η} Σειρά Ασκήσεων

• **Άσκηση 1^{η}:**

α) $T(n) = aT(n/b) + f(n) = 4T(n/2) + n \implies a = 4, b = 2, f(n) = n \implies n^{\log_b a} = n^{\log_2 4} = n^2 \implies f(n) = O(n^{\log_b a - \epsilon})$
Άρα είμαστε στην πρώτη περίπτωση της master μεθόδου και ισχύει: $\underline{T(n) = \Theta(n^2)}$ ...

β) $T(n) = aT(n/b) + f(n) = 4T(n/2) + n^2 \implies a = 4, b = 2, f(n) = n^2 \implies n^{\log_b a} = n^{\log_2 4} = n^2 \implies f(n) = \Theta(n^{\log_b a})$
Άρα είμαστε στην δεύτερη περίπτωση της master μεθόδου και ισχύει: $\underline{T(n) = \Theta(n^2 \log n)}$ ...

γ) $T(n) = aT(n/b) + f(n) = 4T(n/2) + n^3 \implies a = 4, b = 2, f(n) = n^3 \implies n^{\log_b a} = n^{\log_2 4} = n^2 \implies f(n) = \Omega(n^{\log_b a + \epsilon})$
Άρα είμαστε στην τρίτη περίπτωση της master μεθόδου και ισχύει: $\underline{T(n) = \Theta(n^3)}$ ...
Αφού επίσης είναι: $a \cdot f\left(\frac{n}{b}\right) \leq \delta \cdot f(n) \implies 4 \cdot \frac{n^3}{8} \leq \delta \cdot n^3 \implies \frac{1}{2} n^3 \leq \delta \cdot n^3 \implies \frac{1}{2} < \delta < 1$.

• **Άσκηση 2^{η}:**

Αλγόριθμος Α: $T(n) = 7T(n/2) + n^2 \implies \log_2 7 \approx 2.81 > 2 \implies T(n) = \Theta(n^{\log_2 7})$
Αλγόριθμος Α′: $T'(n) = aT'(n/4) + n^2 \implies n^{\log_{b'} a'} = n^{\log_4 a} \implies T'(n) = \Theta(n^{\log_4 a})$

Πρέπει: $T'(n) < T(n) \Rightarrow \Theta(n^{\log_4 a}) < \Theta(n^{\log_2 7}) \Rightarrow \log_4 a < \log_2 7 \Rightarrow \frac{\log_2 a}{\log_2 4} < \log_2 7 \Rightarrow \log_2 a < 2 \log_2 7 \Rightarrow \log_2 a < \log_2 49$
Άρα: $a < 49 \implies \underline{a_{max} = 48}$ (ώστε Α′ ασυμπτωτικά γρηγορότερος από τον αλγόριθμο Α ...)

• **Άσκηση 3^{η}:**

a) $T(n) = 2T(n/2) + n^3 \implies a = b = 2, f(n) = n^3 \implies n^{\log_b a} = n^{\log_2 2} = n \implies f(n) = \Omega(n^{\log_b a + \epsilon}) \implies \underline{T(n) = \Theta(n^3)}$
$(master\,method\,\#3) \quad af(\frac{n}{b}) \leq \delta f(n) \implies 2\frac{n^3}{8} \leq \delta n^3 \implies \frac{1}{4} n^3 \leq \delta n^3 \implies \frac{1}{4} < \delta < 1$

b) $T(n) = T(9n/10) + n \implies a = 1, b = 10/9, f(n) = n \implies n^{\log_{10/9} 1} = n^0 = 1 \implies f(n) = \Omega(n^{\log_b a + \epsilon}) \implies \underline{T(n) = \Theta(n)}$
$(master\,method\,\#3) \quad af(\frac{n}{b}) \leq \delta f(n) \implies 1\frac{9n}{10} \leq \delta n \implies \frac{9}{10} < \delta < 1$

c) $T(n) = 16T(n/4) + n^2 \implies a = 16, b = 4, f(n) = n^2 \implies n^{\log_4 16} = n^2 \implies f(n) = \Theta(n^{\log_b a} \log^k n) \implies \underline{T(n) = \Theta(n^2 \log n)}$
$(master\ method\ \#2)$

d) $T(n) = 7T(n/3) + n^2 \implies a = 7, b = 3, f(n) = n^2 \implies n^{\log_3 7} = n^{1.77} \implies f(n) = \Omega(n^{\log_b a + \epsilon}) \implies \underline{T(n) = \Theta(n^2)}$
$(master\,method\,\#3) \quad af(\frac{n}{b}) \leq \delta f(n) \implies \frac{7}{9} n^2 \leq \delta n^2 \implies \frac{7}{9} < \delta < 1$

e) $T(n) = 7T(n/2) + n^2 \implies a = 7, b = 2, f(n) = n^2 \implies n^{\log_2 7} \approx n^{2.81} \implies f(n) = O(n^{\log_b a - \epsilon}) \implies \underline{T(n) = \Theta(n^{\log_2 7})}$
$(master\ method\ \#1)$

f) $T(n) = T(\sqrt{n}) + 1$ ...
∗ Θέτω $m = \log n \implies n = 2^m$ και $\sqrt{n} = n^{1/2} = 2^{m/2}$ άρα $T(2^m) = T(2^{m/2}) + 1$ έτσι $S(m) = T(2^m)$ οπότε $S(m) = S(m/2) + 1$ και $a = 1, b = 2, f(m) = 1$ άρα $m^{\log_2 1} = 1$ οπότε $S(m) = \Theta(\log m)$ και αφού $m = \log n$ είναι $\underline{T(n) = \Theta(\log(\log n))}$ από (*master method* #2) ...

g) $T(n) = T(n-1) + n$ ...
∗ Έχουμε $T(n) = T(n-1) + n \implies T(n) = T(n-2) + (n-1) + n \implies T(n) = T(n-3) + (n-2) + (n-1) + n \implies ... \implies T(n) = 1 + 2 + 3 + ... + n \implies T(n) = \frac{n(n+1)}{2}$ αφού είναι το άθροισμα ν ακεραίων ... έτσι $\underline{T(n) = \Theta(n^2)}$ ...

• **Άσκηση 4η:**

∗ Έστω ότι έχουμε τα εξής αντικείμενα με βάρος και αξία: Α-1-3, Β-3-8, Γ-2-4. Ο άπληστος αλγόριθμος του σακιδίου με μέγιστο συνολικό βάρος 3, θα επιλέξει αρχικά το αντικείμενο Α με βάρος 1 και αξία 3. Έπειτα, η μόνη διαθέσιμη επιλογή είναι το αντικείμενο Γ με βάρος 2 και αξία 4. Άρα απέκτησε συνολικά 2 αντικείμενα με βάρος 3 και αξία 7. Όμως, εάν δεν επέλεγε εκείνα με το μικρότερο βάρος, θα μπορούσε να διαλέξει το αντικείμενο Β που έχει (ίδιο) βάρος 3 αλλά αξία 8 που είναι μεγαλύτερη. Τελικά, δεν συνεπάγεται αναγκαστικά μια βέλτιστη λύση ... άθροισμα ν ακεραίων ... έτσι $\underline{T(n) = \Theta(n^2)}$ ...

• **Άσκηση 5η:**

∗ Διαδικασία διαίρει και βασίλευε για τον υπολογισμό του Κ μεγαλύτερου στοιχείου σε μια σειρά ακεραίων:
- Βήμα 1: Επιλέγουμε για οδηγό ένα τυχαίο στοιχείο του πίνακα.
- Βήμα 2: Κατανέμουμε τον πίνακα με τον εξής τρόπο:
- - Όλα τα στοιχεία που είναι μεγαλύτερα ή ίσα με τον οδηγό μετακινούνται στα αριστερά του.
- - Όλα τα στοιχεία που είναι μικρότερα από τον οδηγό μετακινούνται στα δεξιά του.
- - Έτσι, ο οδηγός τοποθετείται στην τελική και σωστή του θέση μέσα στον πίνακα.
- Βήμα 3: Εφαρμόζουμε την αναδρομή συγκρίνοντας την τρέχουσα θέση του οδηγού με την αναμενόμενη θέση:
- - Αν ο οδηγός βρίσκεται ακριβώς στην αναμενόμενη θέση, τότε είναι το στοιχείο που ψάχνουμε και επιστρέφουμε την τιμή του.
- - Αν η θέση του οδηγού είναι μεγαλύτερη από την αναμενόμενη, το στοιχείο που ψάχνουμε βρίσκεται σίγουρα στο αριστερό τμήμα, άρα, επαναλαμβάνουμε τη διαδικασία αναδρομικά μόνο για τον υποπίνακα από την αρχή έως την προηγούμενη θέση του οδηγού.
- - Αν η θέση του οδηγού είναι μικρότερη από την αναμενόμενη, το στοιχείο βρίσκεται στο δεξιό τμήμα, οπότε, επαναλαμβάνουμε τη διαδικασία αναδρομικά μόνο για τον υποπίνακα από την επόμενη θέση του οδηγού έως το τέλος του πίνακα.

∗ Ασυμπτωτικής Πολυπλοκότητας Χρόνου:
- Μέση Περίπτωση: $O(N)$
- - Αφού σε κάθε βήμα ο πίνακας χωρίζεται περίπου στη μέση και επεξεργαζόμαστε μόνο το ένα από αυτά τα δύο κομμάτια, αγνοώντας εντελώς το άλλο.
- Καλύτερη Περίπτωση: $O(N)$
- - Συμβαίνει όταν ο πρώτος οδηγός που επιλέγουμε καταλήγει με την πρώτη ακριβώς στη θέση που αναζητούμε, άρα, διατρέχουμε τον πίνακα μόνο μία φορά για την κατανομή και ο αλγόριθμος τερματίζει.
- Χειρότερη Περίπτωση: $O(N^2)$
- - Επειδή σε κάθε βήμα της αναδρομής, ο οδηγός που επιλέγουμε τυχαίνει να είναι το μεγαλύτερο ή το μικρότερο στοιχείο του υποπίνακα, και έτσι η διαδικασία κατανομής δεν χωρίζει τον πίνακα στα δύο, αλλά μειώνει το μέγεθός του μόνο κατά 1 στοιχείο τη φορά, αναγκάζοντάς μας να σαρώνουμε σχεδόν όλα τα στοιχεία ξανά και ξανά ...

• **Άσκηση 6η:**

1. Οι δύο εκδοχές του Knapsack Problem είναι το The Fractional Knapsack Problem και το The 0/1 Knapsack Problem ...

2. Οι διαφορές τους είναι οι εξής:
∗ The Fractional Knapsack Problem:
- - Αντικείμενα: Διαιρετά (επιτρέπονται κλάσματα).
- - Επιλογές: Μπορούμε να επιλέξουμε ένα μέρος του αντικειμένου, αν ολόκληρο δεν χωράει στο σακίδιο.
- - Τύπος Προγραμματισμού: Άπληστος Αλγόριθμος (Greedy Algorithm).
∗ The 0/1 Knapsack Problem:
- - Αντικείμενα: Αδιαίρετα (δεν επιτρέπονται κλάσματα/μισά).
- - Επιλογές: Το αντικείμενο είτε επιλέγεται ολόκληρο είτε μένει εκτός.
- - Τύπος Προγραμματισμού: Δυναμικός Προγραμματισμός (Dynamic Programming).

• **Άσκηση 7η:**

∗ Algorithm 01Knapsack(S, W):
1. Δεδομένα: $W = 5$ και $S = \{(2,3),(3,4),(4,5),(5,6)\}$
2. Αρχικοποίηση: $A[W+1]$; και $B[W+1] = [0,0,0,0,0,0]$;
3. Επαναληπτική διαδικασία:
α. Βήμα $k = 1$ και στοιχείο 1: $(w_1 = 2, b_1 = 3)$:
- - Αντιγράφουμε τον B στον A: $A = [0,0,0,0,0,0]$;
- - Ελέγχουμε από $w = w_1$ έως W (2-5):
- - - $w = 2$: $A[2-2]+3 = A[0]+3 = 0+3 = 3 > A[2] \implies B[2] = 3$
- - - $w = 3$: $A[3-2]+3 = A[1]+3 = 0+3 = 3 > A[3] \implies B[3] = 3$
- - - $w = 4$: $A[4-2]+3 = A[2]+3 = 0+3 = 3 > A[4] \implies B[4] = 3$
- - - $w = 5$: $A[5-2]+3 = A[3]+3 = 0+3 = 3 > A[5] \implies B[5] = 3$
- - Τελικά: $B = [0,0,3,3,3,3]$;
β. Βήμα $k = 2$ και στοιχείο 2: $(w_2 = 3, b_2 = 4)$:
- - Αντιγράφουμε τον B στον A: $A = [0,0,3,3,3,3]$;
- - Ελέγχουμε από $w = w_2$ έως W (3-5):
- - - $w = 3$: $A[3-3]+4 = A[0]+4 = 0+4 = 4 > A[3] \implies B[3] = 4$
- - - $w = 4$: $A[4-3]+4 = A[1]+4 = 0+4 = 4 > A[4] \implies B[4] = 4$
- - - $w = 5$: $A[5-3]+4 = A[2]+4 = 3+4 = 7 > A[5] \implies B[5] = 7$
- - Τελικά: $B = [0,0,3,4,4,7]$;
γ. Βήμα $k = 3$ και στοιχείο 3: $(w_3 = 4, b_3 = 5)$:
- - Αντιγράφουμε τον B στον A: $A = [0,0,3,4,4,7]$;
- - Ελέγχουμε από $w = w_3$ έως W (4-5):
- - - $w = 4$: $A[4-4]+5 = A[0]+5 = 0+5 = 5 > A[4] \implies B[4] = 5$
- - - $w = 5$: $A[5-4]+5 = A[1]+5 = 0+5 = 5 \leq A[5] \implies B[5] = 7$
- - Τελικά: $B = [0,0,3,4,5,7]$;
δ. Βήμα $k = 4$ και στοιχείο 4: $(w_4 = 5, b_4 = 6)$:
- - Αντιγράφουμε τον B στον A: $A = [0,0,3,4,5,7]$;
- - Ελέγχουμε από $w = w_4$ έως W (5):
- - - $w = 5$: $A[5-5]+6 = A[0]+6 = 0+6 = 6 \leq A[5] \implies B[5] = 7$
- - Τελικά: $B = [0,0,3,4,5,7]$;
4. Αποτέλεσμα: Το μέγιστο όφελος είναι $B[W] = B[5] = 7$.

• **Άσκηση 8η:**

α) Και τα δύο προβλήματα Knapsack (το Διακριτό και το Κλασματικό) παρουσιάζουν την ιδιότητα της βέλτιστης υποδομής, αφού αν αφαιρέσουμε ένα στοιχείο από την τελική βέλτιστη επιλογή, τα στοιχεία που απομένουν αποτελούν εγγυημένα τη βέλτιστη επιλογή για τον εναπομείναντα χώρο του σάκου, δηλαδή η βέλτιστη λύση του αρχικού προβλήματος εμπεριέχει αναγκαστικά τις βέλτιστες λύσεις των υποπροβλημάτων του.

β1. Το Διακριτό πρόβλημα δεν υποστηρίζει την ιδιότητα της άπληστης επιλογής, αφού η επιλογή του αντικειμένου με τον καλύτερο λόγο αξίας/βάρους μπορεί να αφήσει ανεκμετάλλευτο κενό χώρο στον σάκο, αποκλείοντας άλλους συνδυασμούς ολόκληρων αντικειμένων που θα τον γέμιζαν ακριβώς και θα απέφεραν μεγαλύτερη συνολική αξία.

β2. Το Κλασματικό πρόβλημα παρουσιάζει και υποστηρίζει την ιδιότητα της άπληστης επιλογής, επειδή μας επιτρέπεται να κόψουμε τα αντικείμενα, επιλέγοντας, έτσι, διαρκώς το αντικείμενο με τον υψηλότερο λόγο αξίας/βάρους, γεμίζουμε τον σάκο χωρίς να αφήσουμε καθόλου κενό χώρο, εξασφαλίζοντας εγγυημένα το μέγιστο δυνατό κέρδος.

• **Άσκηση 9η:**

```
Αλγόριθμος K-WAY-MERGE ( lists[ ] ): // list lists with k sorted lists.
  result = [ ]
  min_heap = EmptyMinHeap()
  Για κάθε i από 0 έως k-1:
    Αν η lists[i] δεν είναι άδεια:
      min_heap = insert ( τιμή: lists[i][0], λίστα_id: i, στοιχείο_id: 0 )
  Όσο ο min_heap δεν είναι άδειος:
    min_element = min_heap.Min()
    Πρόσθεσε το min_element.τιμή στον result·
    λίστα_id, επόμενο_id = min_element.λίστα_id, min_element.στοιχείο_id + 1
    Αν το επόμενο_id < μήκος της lists[λίστα_id]:
      επόμενη_τιμή = lists[λίστα_id][επόμενο_id]
      min_heap = insert ( τιμή: επόμενη_τιμή, λίστα_id: λίστα_id, στοιχείο_id: επόμενο_id) )
  Επέστρεψε τον result[ ] ...
```

• **Άσκηση 10η:**

```
* Αλγόριθμος 3-HUFFMAN ( X ): // String X with d distinct characters.
  Υπολόγισε τη συχνότητα f(c) για κάθε χαρακτήρα c του X
  Q = EmptyPriorityQueue()
  Για κάθε χαρακτήρα c στο X:
    Δημιούργησε ένα τριαδικό δέντρο T με έναν κόμβο για το c
    Q = insert ( δέντρο: T, κλειδί: f(c) )
  Αν το len(Q) είναι άρτιος αριθμός:
    Δημιούργησε ένα τριαδικό δέντρο T_dummy για το null
    Q = insert ( δέντρο: T_dummy, κλειδί: 0 )
  Όσο το len(Q) > 1:
    (f1, T1) = Q.remove_min()
    (f2, T2) = Q.remove_min()
    (f3, T3) = Q.remove_min()
    Δημιούργησε νέο τριαδικό δέντρο T με παιδιά τα T1, T2 και T3
    Q = insert ( δέντρο: T, κλειδί: f1 + f2 + f3 )
  (f, T) = Q.remove_min()
  Επέστρεψε το δέντρο T ...
```

LaTeX